Fully Connected Network (MLP)

Hyperparameter choices

Hidden layers

layers, # neurons per layer

Hidden layers learn progressively more complex combinations of features.
Each neuron receives all inputs from the previous layer but with different weights.
For example, in the 1st hidden layer one neuron might respond strongly to top-left pixels, while another detects the bottom curve of a digit. The 2nd hidden layer then combines these patterns, with one neuron recognizing loops and another identifying straight lines.

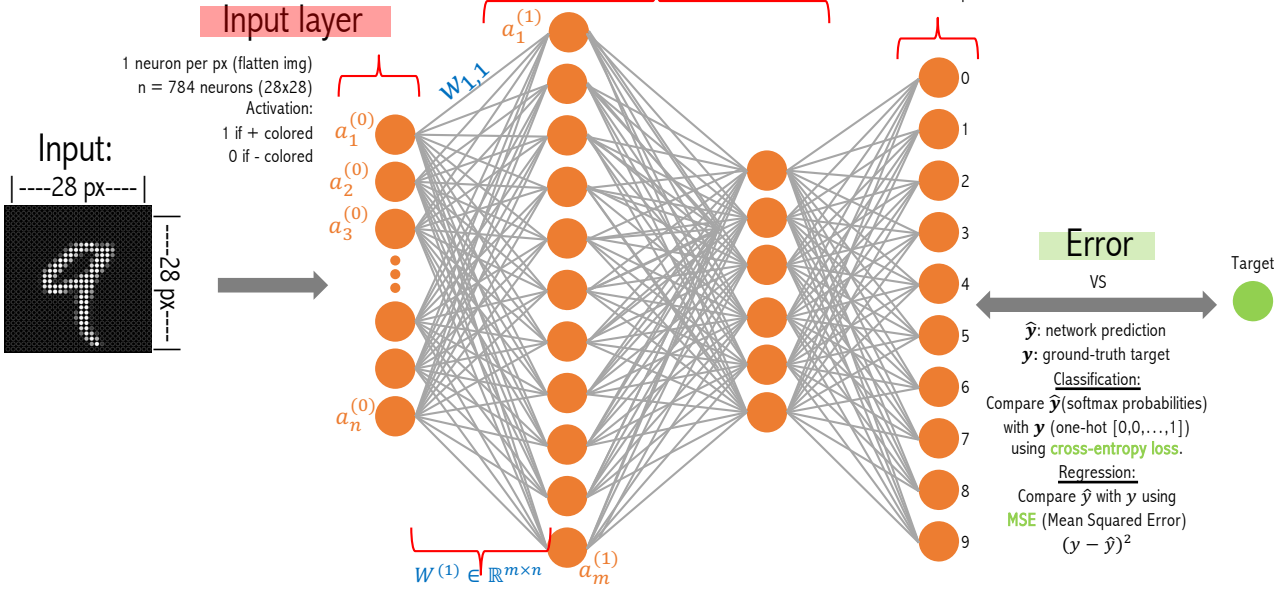
Last layer

Classification:
1 neuron per class
Activation = probability (after softmax) that the input belongs to that class
Regression:
1 neuron (a value) or more if predicting multiple continuous values

Input layer

1 neuron per px (flatten img)
n = 784 neurons (28x28)
Activation:
1 if + colored
0 if - colored

Input:
|----28 px----|
|----28 px----|



Error

VS
Target

$\hat{y}$: network prediction
y: ground-truth target
Classification:
Compare $\hat{y}$(softmax probabilities) with y (one-hot [0,0,…,1]) using cross-entropy loss.
Regression:
Compare $\hat{y}$ with y using MSE (Mean Squared Error)
$(y - \hat{y})^2$

Minimizing the error

- The loss function tells us how wrong the network is.
- The parameters of the network are W and b.
- To make the network better, we want to find the values W and b that minimize the loss function.
- The gradient of the loss function w.r.t. W and b tells us how the loss would change if we changed that parameter. It points in the direction of the steepest increase, so to reduce the loss we move in the opposite direction.
- Backpropagation is the algorithm used to compute the gradients.

Once we have the gradients, an optimizer decides how to update the parameters. E.g. SGD or Adam.

SGD (Stochastic Gradient Descent) STEPS:
(stochastic = use random subsets of data (batches), not the whole dataset, to compute gradients → faster but noisier updates)

0. Initialize W and b (e.g. randomly)
1. Forward pass to compute predictions
2. Compute the loss with the selected loss function
3. Backpropagation: Compute the gradients of the loss w.r.t. W and b
4. Update parameters (SGD in this case):

$w \leftarrow w - \eta \frac{dL}{dw}$, $b \leftarrow b - \eta \frac{dL}{db}$, η = learning rate

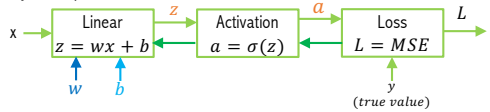
5. Repeat for many steps (epochs) until convergence. One epoch is one iteration over all training data.

- Number of parameter updates= $\frac{total\ \#\ samples\ N}{size\ of\ batch} \times \#\ epochs$

Backpropagation

- Algorithm to compute gradients step by step using the chain rule.
- In one backward pass, it computes gradients w.r.t. all parameters.
- During the forward pass, save intermediate results (z, a, L) which we reuse during backpropagation → efficient

Toy example:



$Then\ \frac{dL}{dw} = \frac{dL}{da}\frac{da}{dz}\frac{dz}{dw}$

Each neuron $a_i^{(l)}$ holds a number (output of a function)

Weights:

How much importance this neuron in the 2nd (or +) layer is picking up on from each of the neurons in the previous layer.
There is one w value per connection (edge).

Bias:

How high the weighted sum needs to be before the neurons start getting meaningfully active. One per neuron.

Vector form:

Activation of previous layer

$$a^{(1)} = \sigma(W^{(1)} a^{(0)} + b^{(1)})$$

Activation function

Matrix form:

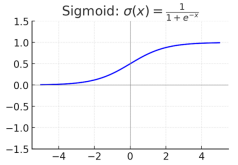
$$\begin{bmatrix} a_1^{(1)} \\ \vdots \\ a_m^{(1)} \end{bmatrix} = \sigma\left(\begin{bmatrix} w_{1,1} & \cdots & w_{1,n} \\ \vdots & \ddots & \vdots \\ w_{m,1} & \cdots & w_{m,n} \end{bmatrix} \begin{bmatrix} a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_m^{(1)} \end{bmatrix} \right)$$

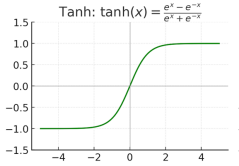
Example for activation for layer 1, neuron 1:

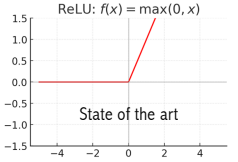
$$a_1^{(1)} = \sigma\left(w_{1,1}\, a_1^{(0)} + w_{1,2}\, a_2^{(0)} + \cdots + w_{1,n}\, a_n^{(0)} + b_1^{(1)} \right)$$

Activation functions:

Intermediate layers:
Activation functions introduce nonlinearity, which lets neural networks model complex patterns. They are differentiable and nonlinear.
Last layer:
- Regression: linear
- Classification: softmax (probability distribution over classes)







Made by: Laura Jimena Tagle M
v.1.0